

Tutorial: Multi-objective spatial optimization of land use

Jessica Ruijsch, Moritz Hildemann, Judith Verstegen, Thomas Gerard, designed in 2020, updated in 2025

1. Introduction

The goal of this tutorial is to show you how to apply an optimization algorithm to non-spatial problems and how you can extend it to solve spatial problems, in this case to optimize land use for a case study area in Brazil.

An optimization algorithm tries to find the most optimal solution(s) considering one or more objectives. For example, you want to maximize the range and acceleration (**objectives**) of an electric car by changing the battery size and the size of the wheels (**control variables**) but without exceeding a certain mass of the car (**constraint**). The objectives, variables and constraints together form the optimization **problem**. When there are multiple, conflicting objectives, as it often the case, there is no single optimal solution, but a set of optimal solutions together forming a **Pareto front**. This is because there are trade-offs between the objectives.

Optimization algorithms are designed to effectively solve optimization problems. Different types of optimization algorithms exist. The optimization algorithm used in this tutorial is an evolutionary algorithm called NSGA-II (Non-dominated Sorting Genetic Algorithm II) (Figure 1). The algorithm starts with an initial **population** (P). The size of this population is increased by creating **offspring** (Q) through **crossover** and **mutation**. Next, the fitness of each individual is calculated using one or more objective functions. The individuals are sorted using **non-dominated sorting** and **crowding distance sorting**, after which the individuals with the lowest fitness are rejected while the individuals with the highest fitness will form the next **generation** of the population (P). New generations are formed until the **termination criterion** is reached.

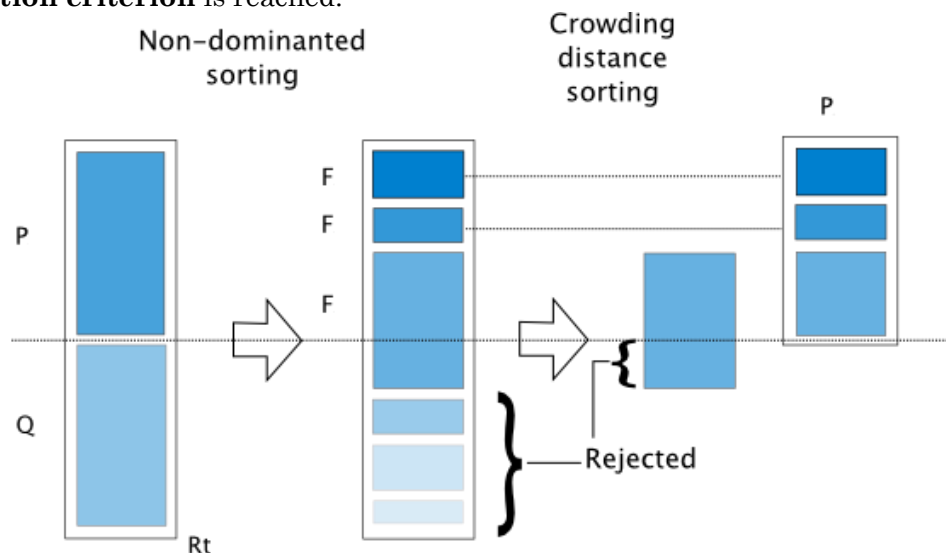


Figure 1. Schematic overview of the NSGA-II algorithm, where P is the population, Q the offspring and Rt the intermediate population. For each individual the fitness F is calculated and sorted with the non-dominated sorting and crowding distance sorting. The part of the population with the highest fitness goes to the next generation while the part with the lowest fitness is rejected. Source: Deb et al. (2002).

In this tutorial, you use an existing non-spatial optimization Python library called **pymoo** (<https://pymoo.org/index.html>) (Blank and Deb, 2020) and create spatial functions to optimize land use in Brazil, maximizing agricultural yield and minimizing CO₂ emissions. The tutorial consists of several chapters where both the pymoo library and the spatial functions are explained step by step. In the end, you should be able to perform a spatial multi-objective optimization of land use.

2. Installation

Install anaconda OR miniconda, if you do not already have it, by using one of the links below:

- <https://www.anaconda.com/> (anaconda)
- <https://docs.conda.io/en/latest/miniconda.html> (miniconda)

Open Anaconda Prompt (Windows) or Terminal (MacOS / Linux). Check if conda is available by typing:

```
conda --version
```

Now, create a new environment with python and Numpy installed via the yml file included in the repository:

```
conda env create -f environment_moso.yml
```

Activate the environment:

```
conda activate moso
```

Next, install the pymoo library:

```
pip install -U pymoo
```

Now, whenever you want to run your scripts, open Anaconda Prompt (Windows) or Terminal (MacOS / Linux), activate the environment, navigate to the location where your script is stored, and then call python, for example:

```
conda activate moso
cd documents\scripts\MOSO_land_use
python myscript.py
```

The repository contains two pre-created Python scripts 'compute_genome.py' and 'spatial_extension_pymoo.py' in a working directory ('wd'), a folder with input data for the initial land use maps and for the objective functions, and a folder with all answer scripts. To make sure that the scripts run correctly, create and keep all scripts for this tutorial in the same folder, for example in the 'wd' folder.

3. Non-spatial multi-objective optimization

This part of the tutorial shows how to use the pymoo library to solve a non-spatial multi-objective optimization problem with the NSGA II algorithm, based on the ‘Getting started’ from the pymoo library on https://pymoo.org/getting_started/index.html. The goal is to get familiar with the pymoo library and use it to solve a simple, non-spatial problem, before adding the spatial extension to the library.

The problem that needs to be optimized is purely mathematical. The problem consists of two objectives, two control variables and two constraints and is defined by:

$$\begin{aligned} \min f_1(x) &= (x_1^2 + x_2^2) \\ \max f_2(x) &= -(x_1 - 1)^2 - x_2^2 \\ \text{s.t.} \quad g_1(x) &= 2(x_1 - 0.1)(x_1 - 0.9) \leq 0 \\ g_2 &= 20(x_1 - 0.4)(x_1 - 0.6) \geq 0 \\ -2 &\leq x_1 \leq 2 \\ -2 &\leq x_2 \leq 2 \end{aligned}$$

Where minimizing f_1 and maximizing f_2 are the objectives, x_1 and x_2 the control variables and g_1 and g_2 the constraints. If you would describe the problem in words: you want to minimize f_1 and maximize f_2 by changing x_1 and x_2 between -2 and 2 while meeting the conditions g_1 and g_2 .

3.1 Define the problem

To solve this problem with the pymoo library, the problem needs to be defined in python. Create a python file called ‘run_nsga2_nonspatial.py’ and open it for editing. First, import the necessary libraries by adding the code below to your script.

```
import numpy as np
from pymoo.model.core.problem import Problem
```

Next, create a class¹ that contains the problem. Create an `__init__` function as described below and replace the ... with the number of variables (`n_var`), number of objectives (`n_obj`), number of constraints (`n_constr`), and upper (`xu`) and lower (`xl`) boundaries corresponding to the problem above. The upper and lower boundaries are defined as an array, with a column for each variable.

```
class MyProblem(Problem):

    def __init__(self):
        super().__init__(n_var= ...,
                          n_obj= ...,
                          n_constr= ...,
                          xl=np.array([ ..., ...]),
                          xu=np.array([ ..., ...]))
```

¹ Python is an object oriented language. A class is a template to create a new object. For more information see: <https://docs.python.org/3/tutorial/classes.html>

The next step is to define the objective functions (f_1 and f_2) and the constraints (g_1 and g_2) in the `_evaluate` method. Copy the code below and fill in the objective functions. While creating the objective functions, consider the following:

- The `_evaluate` method takes X as input. This is a two dimensional NumPy array, where each column is a variable and each row is an individual. This means that x_1 can be called with `X[:, 0]` and x_2 with `X[:, 1]`.
- In python, the syntax of the exponential operator is `**` (thus not `^`), e.g., two to the power four is written as `2**4`.
- The pymoo library can only minimize functions. This means you have to convert f_2 to a minimization function: `max f2 = min -f2`.

The constraints (g_1 and g_2) are already filled in in the code below. Pymoo can only work with constraints that are defined as ≤ 0 . In addition, the constraints are normalized to give them an equal importance in the algorithm; therefore, they look different than in the mathematical notation. For more information on the definition of constraints in pymoo, see https://pymoo.org/getting_started.html.

Pymoo saves the results to the `out` dictionary with the lines `out["F"] = np.column_stack([f1, f2])` and `out["G"] = np.column_stack([g1, g2])`.

You have now defined the problem. Run the code to see if it runs without errors and prints the problem object.

```
def _evaluate(self, X, out, *args, **kwargs):
    f1 = ...
    f2 = ...

    g1 = 2*(X[:, 0]-0.1) * (X[:, 0]-0.9) / 0.18
    g2 = - 20*(X[:, 0]-0.4) * (X[:, 0]-0.6) / 4.8

    out["F"] = np.column_stack([f1, f2])
    out["G"] = np.column_stack([g1, g2])

problem = MyProblem()
print(problem)
```

3.2 Initialize the algorithm

The next step is to create an algorithm object. This is where the population size, number of offspring and the sampling, crossover and mutation methods are defined. In the same script as before (`run_nsga2_nonspatial.py`), define the algorithm object by copying the code below into the script.

You can choose the population size (`pop_size`), for example 40, and the number of offspring (`n_offsprings`), for example 10, yourself. This can be any number, but remember that a too small population will not find an optimal solution and a too large population will result in a long computation time.

In this example we use a Random sampling, a Simulated Binary crossover and a Polynomial mutation. The explanation of these methods and examples of other methods can be found here: <https://pymoo.org/operators/index.html>.

```

from pymoo.algorithms.moo.nsga2 import NSGA2
from pymoo.operators.crossover.sbx import SBX
from pymoo.operators.mutation.pm import PM
from pymoo.operators.sampling.rnd import FloatRandomSampling

algorithm = NSGA2(
    pop_size = ...,
    n_offsprings = ...,
    sampling = FloatRandomSampling(),
    crossover = SBX(prob=0.9, eta=15),
    mutation = PM(eta=20),
    eliminate_duplicates=True
)

```

Run the code again to see whether it runs without error and proceed to the next paragraph.

3.3 Set termination criterion

The pymoo library allows for different types termination criteria, of which the total number of generations ('n_gen') is the easiest to understand. Other termination criteria can be found here: <https://pymoo.org/interface/termination.html>.

Set the termination criterion by copying the code below into the 'run_nsga2_nonspatial.py' script. Choose the number of generations after which you want the algorithm to stop. This can be any number, but the higher the number of generations the longer the algorithm takes to run, and the lower the less likely it is that the algorithm has reached the optimum.

```

from pymoo.termination import get_termination

termination = get_termination("n_gen", ...)

```

Run the code again to see whether it runs without error and proceed to the next paragraph.

3.4 Optimization

Now, we are ready to perform the actual optimization. The `minimize` method runs the optimization algorithm and saves the results as a result object (`res`). When the optimization has finished, the design space (values of the variables) and objective space (values of the objective functions) of the final population can be accessed in `res.X` and `res.F`, respectively. For more information about the results object and how to access the data in the result object, see <https://pymoo.org/interface/result.html>.

To start the optimization and see the results, add the code below to your 'run_nsga2_nonspatial.py' script and run the entire file.

```

from pymoo.optimize import minimize

res = minimize(problem,
               algorithm,
               termination,
               seed=1,
               save_history=True,

```

```

        verbose=True)

print(res)
print(res.X)
print(res.F)

```

3.5 Visualization

To give more insight in the results, you can visualize the design space and the objective space of the final population. In the visualization of the design space, we plot x_1 on the x-axis and x_2 on the y-axis, where each dot represents an individual and the axes ranges correspond to the constraints. In the visualization of the objective space we plot f_1 on the x-axis and f_2 on the y-axis.

Plot the design space and objective space by adding the code below to your script and run it again. Study the resulting figure (Figure 2 and Figure 3).

```

import matplotlib.pyplot as plt

# Plot the design space
f1, ax1 = plt.subplots(1)
ax1.scatter(res.X[:,0], res.X[:,1], s=30, fc='none', ec='r')
ax1.set_title('design space')
ax1.set_xlabel('x1')
ax1.set_ylabel('x2')
ax1.set_xlim(-2, 2)
ax1.set_ylim(-2, 2)
f1.savefig('design_space.png')

# Plot the objective space
f2, ax2 = plt.subplots(1)
ax2.scatter(res.F[:,0], res.F[:,1], s=30, fc='none', ec='k')
ax2.set_title('objective space')
ax2.set_xlabel('f1')
ax2.set_ylabel('f2')
f2.savefig('objective_space.png')

```

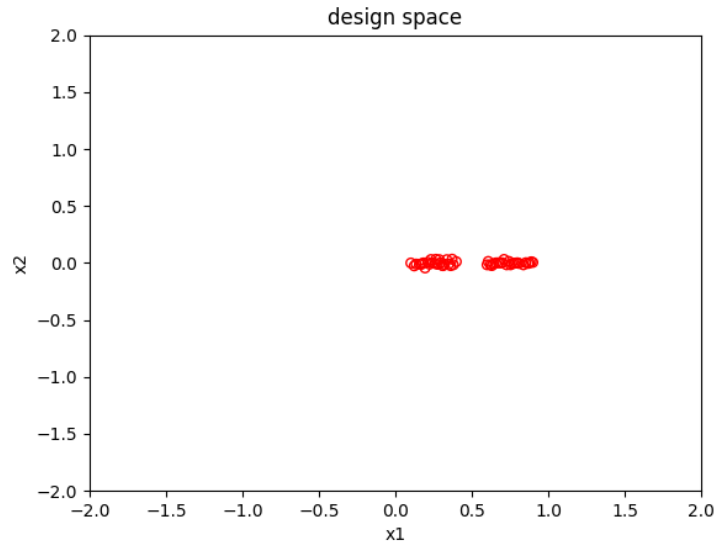


Figure 2. Design space created by the optimization algorithm.

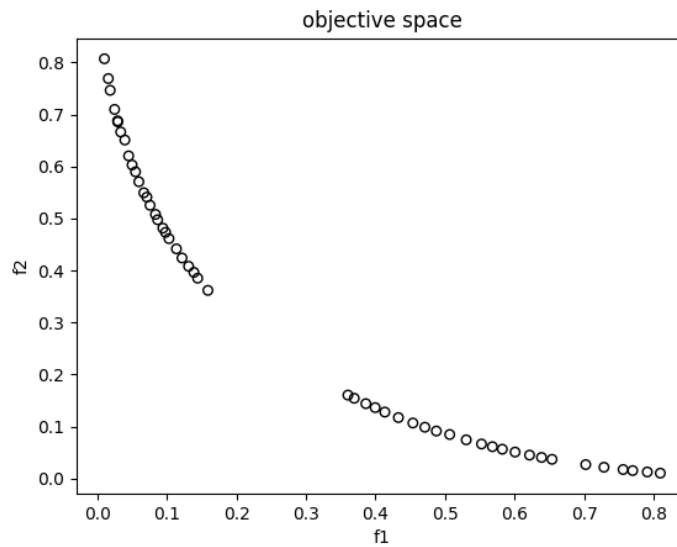


Figure 3. Objective space created by the optimization algorithm.

At this point, you have used a non-spatial multi-objective optimization with the pymoo library. In the remaining part of the tutorial, you will extend the pymoo library with spatial functions to perform a spatial multi-objective optimization. Start by obtaining the required input data in the next chapter.

4. Spatial optimization: input data

In this practical you will use a part of the land use raster of Mato Grosso, a state in the western part of Brazil, from 2017. The raster comes from a series of land use maps from 2001 to 2017 created by Câmara et al. (2019). We have re-projected the raster for this tutorial; you will clip out the study area yourself.

We want to optimize the land use pattern in this area, given two objectives that reflect the desire to get economic return from crop production, as well as the need to limit the CO₂ emissions caused by land use change (through the carbon stored in biomass):

- maximizing the agricultural yield, and
- minimizing the CO₂ emissions.

The control variable in this problem is the land use class of every raster cell in the study area. In our case study, we consider ten possible land use classes:

1. Forest (natural)
2. Cerrado (natural savannah vegetation)
3. Secondary vegetation
4. Soy
5. Sugarcane
6. Fallow / cotton
7. Pasture
8. Water
9. Urban
10. No data

However, to take real-world constraints into account, there are several rules that need to be considered during optimization:

- A raster cell must have exactly one class (between 1 and 10);
- Water, urban area and no data (8, 9, 10) cannot be changed and a cell cannot be changed into water urban area or no data, as this would be too costly in reality; and
- Cerrado and (natural) forest (1, 2) can be changed into agriculture or secondary vegetation, but not the other way around, because we assume that once primary vegetation has been removed, it can never grow back.

4.1 Initial land use map

You will start with reading in the initial land use raster of Mato Grosso. Create a new python script and save it as 'read_data.py'. Import the libraries as shown below and set the default directory. This should be the 'input_data' directory in the tutorial directory.

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.colors import ListedColormap
import matplotlib.patches as mpatches
import pickle

default_directory = ...
```

Although the matplotlib library is meant for plotting, we will use its `imread` function here to open the tiff with the land use raster, to limit the number of libraries you have to install for this tutorial. The function reads the tiff as a numpy array. Copy the code below into your script to load the data and save it to the `landuse_original` variable.


```
# read data from tiff file
landuse_original = plt.imread(default_directory +
                               "/Landuse_maps/mt_2017_v3_1_reprojection.tif")
```

Plot the original land use map using matplotlib imshow and the colormap specified below (html hex strings are used to define the colors). We use a workaround to get a categorical instead of a continuous legend. Make sure to set vmin to 0.5 and vmax to 15.5 when plotting the map. You can save the file with matplotlib imsave; make sure to create a separate folder for that, e.g., 'outputs'. Run the code and study the original land use map that is plotted (Figure 4).

```
# plot original landuse map
f1, ax1 = plt.subplots(1)
cmap1 = ListedColormap(["#b3cc33", "#be94e8", "#10773e", "#eeefce",
                        "#e4a540", "#a4507d", "#c948a2", "#be5b1d",
                        "#f09cde", "#877712", "#614040", "#1b5ee4",
                        "#0cf8c1", "#00000000", "#00000000"])

legend_landusel = [mpatches.Patch(color="#b3cc33", label = 'Cerrado'),
                   mpatches.Patch(color="#be94e8", label = 'Fallow/cotton'),
                   mpatches.Patch(color="#10773e", label = 'Forest'),
                   mpatches.Patch(color="#eeefce", label = 'Pasture'),
                   mpatches.Patch(color="#e4a540", label = 'Soy/corn'),
                   mpatches.Patch(color="#a4507d", label = 'Soy/cotton'),
                   mpatches.Patch(color="#c948a2", label = 'Soy/fallow'),
                   mpatches.Patch(color="#be5b1d", label = 'Soy/millet'),
                   mpatches.Patch(color="#f09cde", label = 'Soy/sunflower'),
                   mpatches.Patch(color="#877712", label = 'Sugarcane'),
                   mpatches.Patch(color="#614040", label = 'Urban area'),
                   mpatches.Patch(color="#1b5ee4", label = 'Water'),
                   mpatches.Patch(color="#0cf8c1", label = 'Secondary veg.'),
                   mpatches.Patch(color="#00000000", label = 'No data')]

im1 = plt.imshow(landuse_original, interpolation='none',
                  cmap=cmap1, vmin = 0.5, vmax = 15.5)
ax1.set_title('Landuse map original')
ax1.set_xlabel('Column #')
ax1.set_ylabel('Row #')
ax1.legend(handles=legend_landusel, bbox_to_anchor=(1.05, 1), loc=2,
           borderaxespad=0.)
plt.imsave(default_directory +
            "/outputs/MatoGrosso_2017_original.tif",
            landuse_original, format='tiff', cmap=cmap1)
plt.show()
```

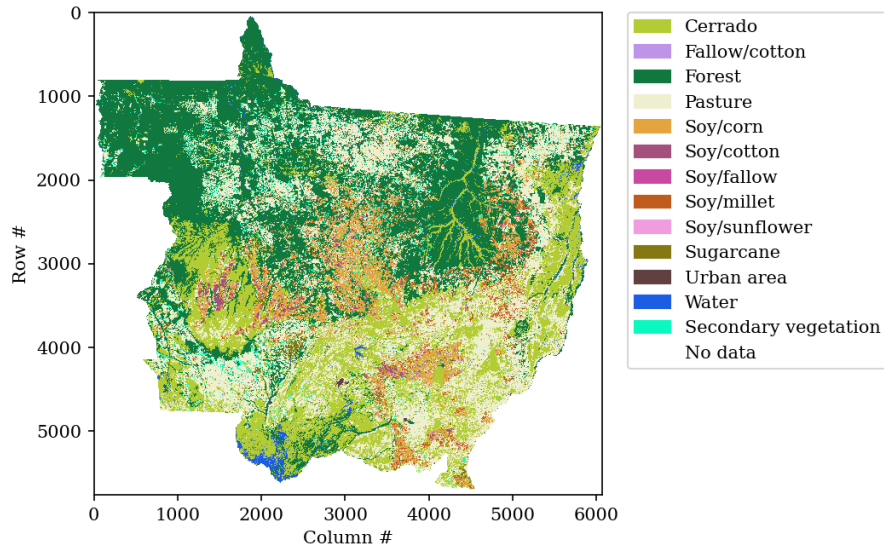


Figure 4. Original land use map of Mato Grosso, Brazil.

4.2 Reclassify the land use map

You see that the original land use map has fourteen classes. Based on our objectives, we have decided to limit the number of classes to ten (see list of classes in introduction), to reduce the size of the problem to solve. Therefore, the `landuse_original` map needs to be reclassified and saved as `landuse_reclass`. Copy the code below into the `'read_data.py'` file. In the code, an example is given of the reclassification of two classes. Complete the code by adding new lines for each land use class. You can use Table 1 to check what should be reclassified into what.

```
# create empty map
rows = landuse_original.shape[0]
cols = landuse_original.shape[1]
landuse_reclass = np.zeros((rows,cols),dtype= 'uint8')

# reclassify landuse map
landuse_reclass[landuse_original == 1] = 2
landuse_reclass[landuse_original == 2] = 6
...
```

Table 1. List of classes for the original and reclassified land use map.

Original		Reclassified	
1	Cerrado	2	Cerrado
2	Fallow/cotton	6	Cotton
3	Forest	1	Forest
4	Pasture	7	Pasture
5	Soy / corn	4	Soy
6	Soy / cotton	4	Soy
7	Soy / fallow	4	Soy
8	Soy / millet	4	Soy
9	Soy / sunflower	4	Soy
10	Sugarcane	5	Sugarcane
11	Urban area	9	Urban area
12	Water	8	Water
13	Secondary vegetation	3	Secondary vegetation
15	No data	10	No data

Define a new figure and the colormap as show below and add the code to plot the reclassified land use map (use a vmin of 0.5 and vmax of 10.5).

```
f2, ax2 = plt.subplots(1)
cmap2 = ListedColormap(["#10773e", "#b3cc33", "#0cf8c1", "#a4507d",
                        "#877712", "#be94e8", "#eeefce", "#1b5ee4",
                        "#614040", "#00000000"])

legend_landuse2 = [mpatches.Patch(color="#10773e", label = 'Forest'),
                   mpatches.Patch(color="#b3cc33", label = 'Cerrado'),
                   mpatches.Patch(color="#0cf8c1", label = 'Secondary veg.'),
                   mpatches.Patch(color="#a4507d", label = 'Soy'),
                   mpatches.Patch(color="#877712", label = 'Sugarcane'),
                   mpatches.Patch(color="#be94e8", label = 'Fallow/cotton'),
                   mpatches.Patch(color="#eeefce", label = 'Pasture'),
                   mpatches.Patch(color="#1b5ee4", label = 'Water'),
                   mpatches.Patch(color="#614040", label = 'Urban'),
                   mpatches.Patch(color="#00000000", label = 'No data')]
```

Run the code and study the reclassified land use map (Figure 5).

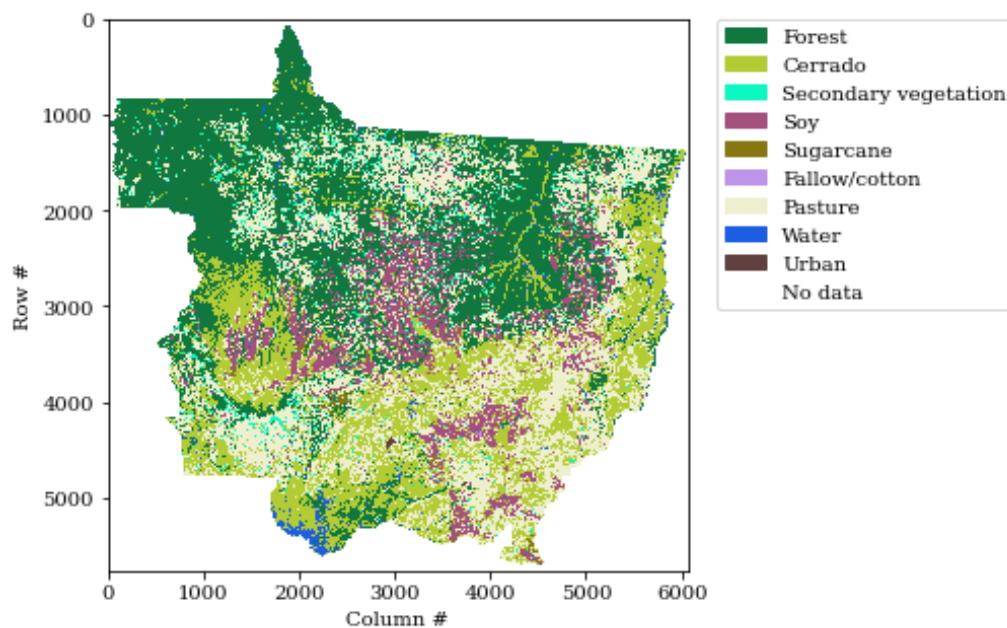


Figure 5. Reclassified land use map of Mato Grosso, Brazil

4.3 Crop the study area

Because the study area is quite large (5764 x 6072 cells), a smaller area of 100 x 100 cells is selected to reduce the computation time of the optimization. Copy the code below into 'read_data.py' to crop the land use map and save the clipped land use map as landuse_map_in.npy. This file is used as input for the optimization algorithm, so make sure to use exactly 'landuse_map_in.npy' as name for the file. Plot the clipped land use map using the same settings as before, run the code, and study the result (Figure 4).

```
landuse_clipped = landuse_reclass[2800:2900,1700:1800]
np.save(default_directory + "/outputs/landuse_map_in.npy", landuse_clipped)
```

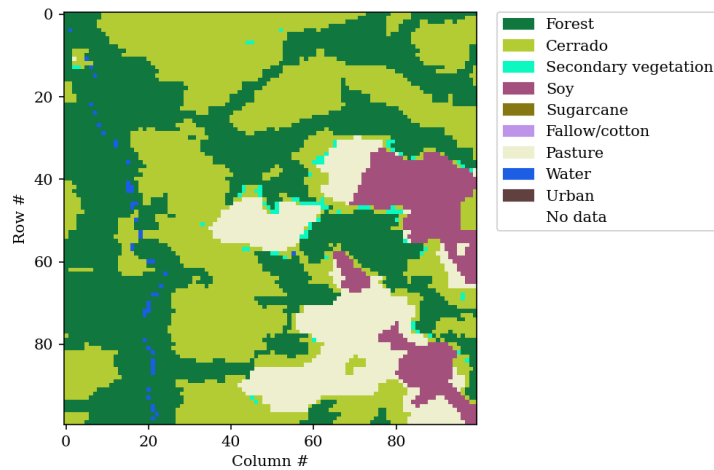


Figure 6. Clipped land use map of Mato Grosso, Brazil.

4.4 Read potential yield maps

To calculate the agricultural yield, which is input to one of the objective functions, for every solution (individual in the population) potential yield rasters of sugarcane, soy, cotton and pasture are required. The cell value in a potential yield raster represents the yield (tonnes/ha) that would be obtained if the crop was planted in that raster cell. These values are computed based on climate and soil variables by the Food and Agriculture Organization (FAO) (Fischer et al., 2012) (<https://web.archive.iiasa.ac.at/Research/LUC/GAEZv3.0/>).

Read the potential yield maps (ASCII files), clip them to the same size as your land use raster, and save the files as pickle (.pkl) files to reduce the computation time during optimization, by copying the code below into the 'read_data.py' file.

```
# read potential yield maps from asc file
sugarcane_pot_yield = np.loadtxt(default_directory +
    "/Objectives/sugarcane_new.asc",
    skiprows=6) [2800:2900,1700:1800]
soy_pot_yield = np.loadtxt(default_directory +
    "/Objectives/soy_new.asc",
    skiprows=6) [2800:2900,1700:1800]
cotton_pot_yield = np.loadtxt(default_directory +
    "/Objectives/cotton_new.asc",
    skiprows=6) [2800:2900,1700:1800]
pasture_pot_yield = np.loadtxt(default_directory +
    "/Objectives/grass_new.asc",
    skiprows=6) [2800:2900,1700:1800]

# save the cropped potential yield maps
with open(default_directory + "/Objectives/sugarcane_potential_yield_example.pkl", 'wb') as output:
    pickle.dump(sugarcane_pot_yield, output, pickle.HIGHEST_PROTOCOL)

with open(default_directory + "/Objectives/soy_potential_yield_example.pkl", 'wb') as output:
    pickle.dump(soy_pot_yield, output, pickle.HIGHEST_PROTOCOL)

with open(default_directory + "/Objectives/cotton_potential_yield_example.pkl", 'wb') as output:
    pickle.dump(cotton_pot_yield, output, pickle.HIGHEST_PROTOCOL)
```

```
with open(default_directory + "/Objectives/pasture_potential_yield_example.  
pkl", 'wb') as output:  
    pickle.dump(pasture_pot_yield, output, pickle.HIGHEST_PROTOCOL)
```

Run the entire script to create all the necessary input data and make sure it runs without errors. Visualize one of the potential yield maps using `matplotlib imshow` and study the plot. You will see that the resolution is very coarse, but it is good enough for an indication of potential yield in the different parts within the study area.

The computation of CO₂ emissions will be based on a biomass value per land use class. Therefore, we do not need additional spatial input data for the second objective. Thus, all required input data are available for the optimization algorithm. Continue to the next chapter, where the objective functions are defined.

5. Spatial optimization: objective functions

This tutorial runs the optimization based on two objectives as defined before: maximize agricultural yield and minimize CO₂ emissions. This chapter explains how to create these objective functions. First, create a new python script called 'calculate_objectives.py' and import the necessary libraries. As mentioned before, make sure all your scripts are in the same folder as the 'compute_genome.py' file (e.g. in 'wd'). Otherwise you will not be able to import the 'create_patch_ID_map' function.

```
import numpy as np
from compute_genome import create_patch_ID_map
```

5.1 Maximize agricultural yield

The total agricultural yield of a certain land use configuration can be calculated using the potential yield rasters. There are four agricultural land use classes: sugarcane, soy, cotton and pasture (grass).

Create a function `calculate_tot_yield` that takes `landuse_map_in`, the potential yield maps and the cell area as input. Loop over all the individuals in the population (right now, we have only one) and calculate the yield of each agricultural class. In the end, calculate the total yield of all agricultural classes by summing the yield of soy, sugarcane, cotton and grass.

An example for soy is given below. Copy the code into the 'calculate_objectives.py' file and make sure you understand the code. Add similar calculations for sugarcane, cotton and pasture inside the function.

```
# calculate the total yield for sugarcane, soy, cotton and pasture
def calculate_tot_yield(landuse_map_in, sugarcane_map, soy_map, cotton_map,
                       pasture_map, cellarea):
    all_yields = []

    # loop over the individuals in the population
    for land_use_map in landuse_map_in:

        # soy
        yield_soy = np.where(land_use_map == 4, soy_map*cellarea, 0)
        tot_yield_soy = np.sum(yield_soy)

        # total yield agriculture is equal to sum of different crops
        tot_yield = tot_yield_soy + ...
        all_yields.append(tot_yield)

    return(np.array(all_yields))
```

The function takes as inputs the land use rasters of all individuals in the population, the potential yield rasters, and the value of the area of a cell and returns an array of objective function values for each land use map in the population.

Add some lines of code to test your function. Hereto, use the current land use and the potential yield rasters. The code below shows example of how to load in land use raster (you need to add an extra dimension because the function expects a population of rasters) and the potential yield raster of sugar cane. The cell area is 250 by 250 meters = 6.25 hectare.

```
# read clipped land use
landuse = np.load(default_directory + "/Landuse_maps/landuse_map_in.npy")
```

```

landuse = [landuse]
print(np.shape(landuse))

# read input data for objectives
with open(default_directory + "/Objectives/sugarcane_potential_yield_example.pkl", 'rb') as output:
    sugarcane_pot_yield = pickle.load(output)

```

Run the script to make sure it runs without error messages. The total yield should be around 25000 tonnes. Afterwards, comment out the testing code and save the script.

5.2 Minimize CO₂ emissions

The second objective is to minimize CO₂ emissions. To keep things simple, the above ground biomass (AGB) is maximized as indication for minimal CO₂ emissions. To illustrate this, when forest is removed for the sake of agriculture, the above ground biomass decreases and is eventually turned into CO₂.

Below, a function `calculate_above_ground_biomass` is defined that calculates the total above ground biomass for each land use configuration. This calculates the area of forest and then the above ground biomass (tonnes) by multiplying it with the average above ground biomass of forest per hectare (Table 2). It does the same for Cerrado.

Copy the code below to the ‘`calculate_objectives.py`’ script and add the calculations for all other land use classes, using Table 2. Note that annual crops have zero biomass, because they are harvested each year. Finally, calculate the total biomass of the land use map by summing over all land use classes.

```

def calculate_above_ground_biomass(landuse_map_in, cellarea):
    # loop over the individuals in the population
    all_emissions = []
    for land_use_map in landuse_map_in:
        # calculate the total area of each land use class
        forest_area = np.count_nonzero(land_use_map == 1)*cellarea
        cerrado_area = np.count_nonzero(land_use_map == 2)*cellarea

        # multiply area of the land use class (ha) with the AGB (tonnes/ha)
        agb_forest = forest_area*300
        agb_cerrado = cerrado_area*48

        # sum over all land use classes
        above_ground_biomass_all = agb_forest + agb_cerrado + ...

        # add to the array with all individuals
        all_emissions.append(above_ground_biomass_all)

    return np.array(all_emissions)

```

Table 2. Values of above ground biomass (tonnes/ha) for each land use class.

		Above ground biomass (tonnes/ha)	Source
1	Forest	300	Eggleston et al. (2006)
2	Cerrado	48	Ribeiro et al. (2011)
3	Secondary vegetation	150	Eggleston et al. (2006)
4	Soy	0	-
5	Sugarcane	16	Kerdan et al. (2019)
6	Fallow / cotton	0	-

7	Pasture	$6.7 * 0.7$	Eggleston et al. (2006)
8	Water	0	-
9	Urban	0	-
10	No data	0	-

As with the previous function, this objective function calculates the total above ground biomass for each individual in the entire population and returns an array of the objective function values of the population. Add some lines of code to test your function. Run the script to check for errors. The total above-ground biomass should be around 8.7 million tonnes. Afterwards, comment out the testing code.

6. Spatial optimization: initial population

At this point, you have created the input files and the objective functions. The next step is to use the pymoo library for optimization. However, because pymoo is non-spatial, you need to create spatial initialization, spatial crossover and spatial mutation functions yourself. It is important that your functions have the same output type and data structure as the original pymoo function to ensure compatibility with the rest of the optimization. The initial population should be a Numpy array of all land use rasters combined.

Let's start with creating an initial population. In pymoo, the initial population is created with the sampling method defined in the algorithm object. If needed, look back at the non-spatial optimization chapter. Create a python script called 'spatial_sampling.py', copy the code below and set the default directory. This code creates a class that calls the initialize_spatial function, which will perform the actual initialization of the population.

```
import numpy as np
from pymoo.model.sampling import Sampling
import initial_population

class SpatialSampling(Sampling):
    def __init__(self, default_dir=None, var_type=np.float) -> None:
        super().__init__()
        self.default_dir = default_dir
        self.var_type = var_type

    def _do(self, problem, n_samples, **kwargs):
        landusemaps_np = initial_population.initialize_spatial(n_samples,
                                                                self.default_dir)
        return landusemaps_np
```

The next step is to create this initialize_spatial function. Create a script called 'initial_population.py' and copy the code below to create the function initialize_spatial.

This function creates an initial population of land use rasters by randomly selecting 30% of the cells and assigning new land use classes to these cells. To make the new maps more realistic, a window average function is applied to the selection probabilities such that larger patches of several cells are altered instead of single cells. Make sure that you understand the code below and run it to check for errors.

```
import numpy as np

# make initial population for genetic algorithm
def initialize_spatial(pop_size, default_directory):

    all_landusemaps = []
    landuse_map_in = np.load(default_directory +
                              "/Landuse_maps/landuse_map_in.npy")

    rows = landuse_map_in.shape[0]
    cols = landuse_map_in.shape[1]

    # iterate to get multiple realizations for the initial population
    for i in range(1, pop_size+1):

        #use uniform distribution to select 30% of the cells
```

```

landuse_map_ini = np.zeros((rows,cols),dtype='uint8')
random_map = np.random.uniform(0.0,1.0,(rows,cols))
random_map_mw = np.zeros((rows,cols))

# take window average of random map to create larger patches
for x in range(0,cols):
    for y in range(0,rows):
        if x == 0 or y == 0 or x == cols-1 or y == rows-1:
            random_map_mw[x,y] = 1.0
        else:
            random_map_mw[y,x] = random_map[y-1:y+2,x-1:x+2].mean()

# 70% of the map remains the current land use
landuse_map_ini = np.where(random_map_mw>=0.3,
                           landuse_map_in,landuse_map_ini)

# 30% of the map will become new
# urban, water and no data will remain the same
landuse_map_ini = np.where(landuse_map_in >= 8,
                           landuse_map_in,landuse_map_ini)

# other land use classes can change into 3, 4, 5, 6 or 7
# choose which land cover type
landuse_map_ini = np.where(landuse_map_ini == 0,
                           np.random.randint(low=3, high=8,size=(rows,cols)),
                           landuse_map_ini)

all_landusemaps.append(landuse_map_ini)
return np.array(all_landusemaps)

```

Now, test your function by generating three initial land use maps and plotting them in a figure with three subplots, as shown in the code below. Inspect the results (see Figure 7). Note that, although larger patches of several cells are altered, they are not altered into a single land use type, because the assignment of a land use type is not per patch. Afterward inspecting the results, comment out the testing code.

```

default_directory = ...

maps = initialize_spatial(3, default_directory)

f, axes = plt.subplots(1,3)
cmap = ListedColormap(["#10773e", "#b3cc33", "#0cf8c1", "#a4507d",
                      "#877712", "#be94e8", "#eeefce", "#1b5ee4",
                      "#614040", "#00000000"])

for amap, ax in zip(maps, axes):
    im = ax.imshow(amap,interpolation='none', cmap=cmap,vmin = 0.5, vmax =
10.5)

plt.colorbar(im, orientation='horizontal')
plt.show()

```

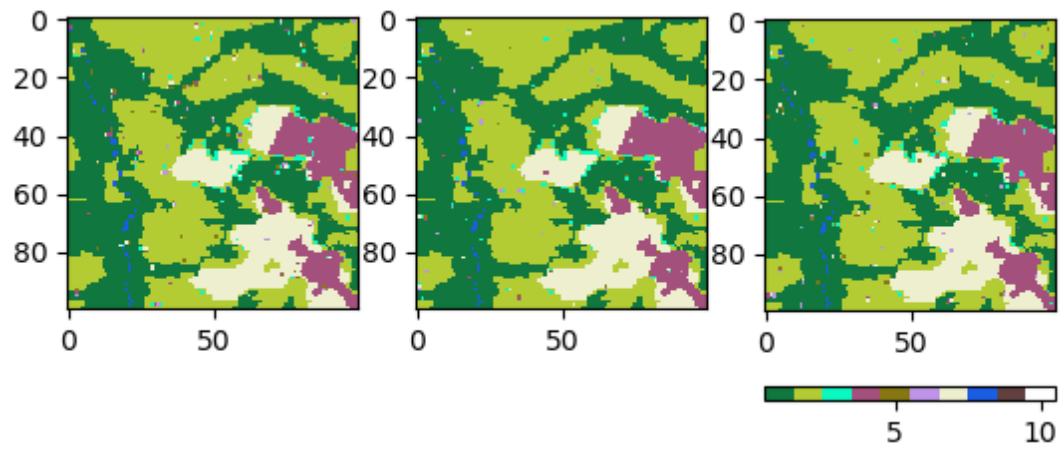


Figure 7: Three sampled initial land use maps.

7. Spatial optimization: crossover

The next step is to create a function for the spatial crossover.

The spatial crossover that will be implemented here is illustrated by Figure 8 and contains the following steps:

1. Start with two land use rasters that are called parent 1 and parent 2.
2. The land use rasters are converted to patch rasters, in which each contiguous area of the same land use class (patch) is numbered. The patches are numbered from top left to bottom right.
3. Land use classes that cannot change (urban and water) get a patch ID of 0.
4. Each patch raster is turned into a genome, which is a string of the patch numbers and the corresponding land use class. This genome is used for the crossover.
5. The genome is cut at a defined number of random places.
6. Child 1 gets part of the genome of parent 1, the rest of the genome remains empty. Child 2 gets the other parts of parent 2, the rest of the genome remains empty.
7. The genomes of child 1 and 2 are converted back into a land use raster, but because part of the genome is empty, part of the land use map will also be empty.
8. The empty parts of child 1 are filled with the corresponding cells of parent 2. The empty parts of child 2 are filled with the corresponding cells of parent 1.
9. The crossover is now finished and there are two children, i.e. two new individuals for the next generation.

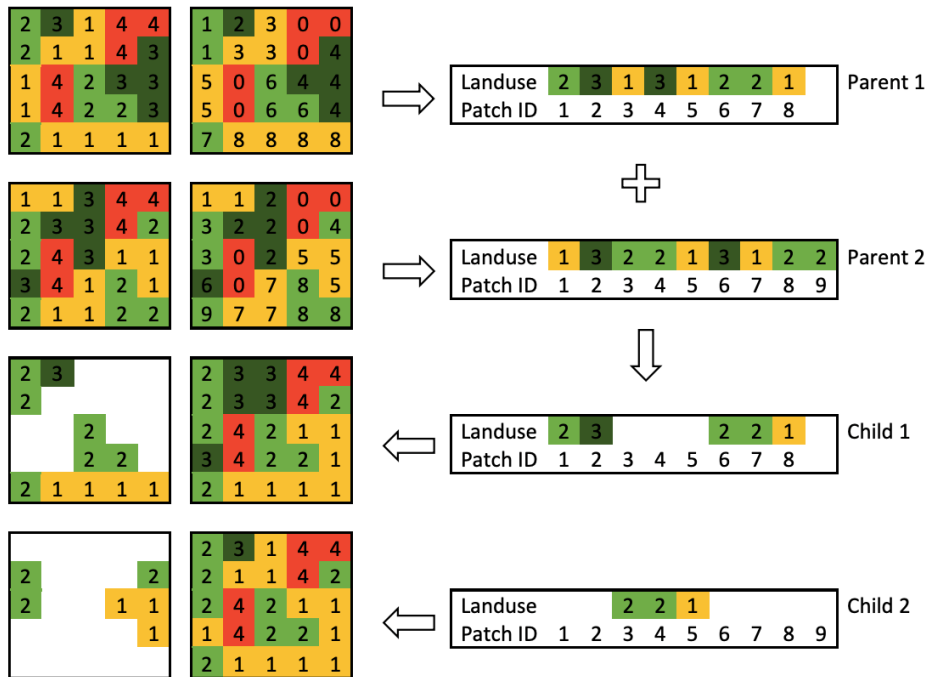


Figure 8. Schematic overview of the spatial crossover implemented in this tutorial.

To perform this spatial crossover in python, we've created a python file called 'spatial_crossover.py' that can be found in the folder 'wd', because it's level of complexity is beyond the scope of this tutorial. It contains a class called `SpatialOnePointCrossover` that is based on pymoo's `Crossover` class. The code follows the steps described above. Two notes for this part of the code:

- To create the patch map and the genome, the function `create_patch_ID_map` from the CoMOLA library (<https://github.com/michstrauch/CoMOLA>) is used (Strauch et al.,

2019). This function is defined in the 'compute_genome.py' file that is also provided to you in the folder 'wd'.

- As before, the structure of the class is important, because it has to fit within the existing pymoo library. The crossover function loops over all the individuals that are selected for cross-over. The function has to return the combined land use maps of the children as a single Numpy array.

8. Spatial optimization: mutation

The last spatial function that needs to be created is the mutation. The mutation used for the tutorial is simple. It randomly changes part of the genome (one patch) into another land use class.

First, it is determined whether mutation occurs in a certain individual. This probability is set by the `prob` variable. If a mutation occurs, the raster is converted into a genome. This process is the same as in the crossover. Next, each patch in the genome is changed into another land use class with a certain probability. This probability is defined with the `point_mutation` variable. In the end, the mutated genome is converted back to a land use raster that becomes part of the population. Figure 9 shows a schematic overview of the mutation.

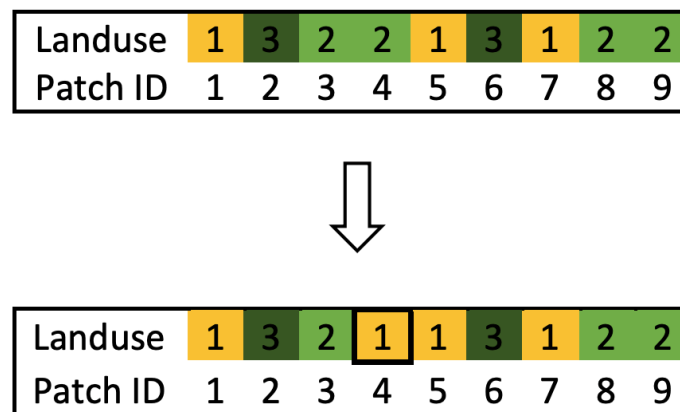


Figure 9. Schematic overview of the spatial mutation implemented in this tutorial.

Create a file called 'spatial_mutation.py' and copy the code below into it. This creates a class `SpatialNPointMutation` that provides a framework for the mutation. The `random_reset_mutation` function performs the mutation.

```
import numpy as np
from pymoo.model.mutation import Mutation
from compute_genome import create_patch_ID_map

# function to randomly change a certain patch
def random_reset_mutation(genome_in, point_mutation_prob):
    genome = list(genome_in)
    for i in range(1, len(genome)):
        if np.random.uniform(0, 1) < point_mutation_prob:
            genome[i] = np.random.randint(low=3, high=8)
    return (genome)

# class that performs the mutation
class SpatialNPointMutation(Mutation):

    def __init__(self, prob=None, point_mutation_probability=0.01):
        super().__init__()
        self.prob = prob
        self.point_mutation_probability = point_mutation_probability

    def _do(self, problem, X, **kwargs):
```

```

shape_landusemaps = [X[0].shape[0], X[0].shape[1]]
rows = shape_landusemaps[0]
cols = shape_landusemaps[1]
offspring = []

# loop over individuals in population
for i in X:
    # perform mutation with certain probability
    if np.random.uniform(0, 1) < self.prob:
        # get genome, where class 8 and 9 are static (not included)
        patches, genome = create_patch_ID_map(i, 0, [8, 9], "True")
        # perform mutation
        mutated_genome = random_reset_mutation(genome,
                                                self.point_mutation_probability)

        # go back to land use map
        mutated_individual = patches
        for x in range(0, cols):
            for y in range(0, rows):
                if mutated_individual[x, y] != 0:
                    mutated_individual[x, y] = \
                        mutated_genome[mutated_individual[x, y] - 1]
                else:
                    mutated_individual[x, y] = i[x,y]
        mutated_individual = np.where(mutated_individual == 0, i,
                                      mutated_individual)
        offspring.append(mutated_individual)
    # if no mutation
    else:
        offspring.append(i)
offspring = np.array(offspring)
return offspring

```

Note that the same 'compute_genome.py' file as in the crossover is used to convert the land use map into a genome before mutation can be applied. Make sure you understand the mutation function before you move on to the next chapter.

9. Spatial optimization: run algorithm

9.1 Read input data

Now that all spatial functions are created, it is time to run the spatial optimization. Create a new python file called 'run_nsga2_spatial.py'.

Import the necessary libraries, set the default directory and read the input data by copying the code below into the 'run_nsga2_spatial.py' file. Run the code.

```
import numpy as np
import pickle
from initial_population import initialize_spatial

default_directory = ...

cell_area = 2.5 * 2.5 # in hectares

# read input data for objectives
with open(default_directory + "/Objectives/sugarcane_potential_yield_example.pkl", 'rb') as output:
    sugarcane_pot_yield = pickle.load(output)

with open(default_directory + "/Objectives/soy_potential_yield_example.pkl", 'rb') as output:
    soy_pot_yield = pickle.load(output)

with open(default_directory + "/Objectives/cotton_potential_yield_example.pkl", 'rb') as output:
    cotton_pot_yield = pickle.load(output)

with open(default_directory + "/Objectives/pasture_potential_yield_example.pkl", 'rb') as output:
    pasture_pot_yield = pickle.load(output)
```

9.2 Run the algorithm

As with the non-spatial problem, the problem, algorithm and termination need to be set before the optimization can be started. Copy the necessary code from 'run_nsga2_nonspatial.py' into the 'run_nsga2_spatial.py' script and think about which part of the code needs to be changed for this spatial problem.

Start with the MyProblem class. In the __init__, set the number of variables (100), number of objectives, number of constraints and the upper and lower boundary. The number of variables is set to 100, to correspond to one dimension of the 100x100 land use raster. Because the upper and lower boundary are not used in the spatial algorithm, you do not have to change them.

```
from pymoo.util.misc import stack
from pymoo.model.problem import Problem
from calculate_objectives import calculate_tot_yield, calculate_above_ground_biomass, calculate_landuse_patches

class MyProblem(Problem):
```



```

# define the number of variables etc.
def __init__(self):
    super().__init__(n_var=100,          # nr of variables
                     n_obj=2,           # nr of objectives
                     n_constr=0,        # nr of constraints
                     xl=0.0,            # lower boundaries
                     xu=1.0)           # upper boundaries

```

In `_evaluate`, define the objective functions (f1 and f2) by calling `calculate_tot_yield` and `calculate_above_ground_biomass` with the required input. Think about whether you need to minimize or maximize these and sign the functions needed as a consequence. In this case, the constraints are incorporated into the initialization, crossover and mutation and not defined by the Problem class, so you do not need to define `g1` and `g2`. Run the script to test if the problem definition has worked.

```

# define the objective functions
def _evaluate(self, X, out, *args, **kwargs):
    f1 = -calculate_tot_yield(X[:,], sugarcane_pot_yield, soy_pot_yield,
                             cotton_pot_yield, pasture_pot_yield, cell_area)
    f2 = -calculate_above_ground_biomass(X[:,], cell_area)

    out["F"] = np.column_stack([f1, f2])

problem = MyProblem()

```

The next step is to initialize the algorithm. Add the code below to the script. Set the population size to 70, the number of offspring to 10, and the sampling, crossover and mutation to their spatial variants. Set `eliminate_duplicates` to False.

```

from pymoo.algorithms.moo.nsga2 import NSGA2
from spatial_sampling import SpatialSampling
from spatial_crossover import SpatialOnePointCrossover
from spatial_mutation import SpatialNPointMutation

algorithm = NSGA2(
    pop_size = 70,
    n_offsprings = 10,
    sampling = SpatialSampling(default_directory),
    crossover = SpatialOnePointCrossover(n_points=3),
    mutation = SpatialNPointMutation(prob = 0.1,
                                     point_mutation_probability = 0.1),
    eliminate_duplicates=False)

```

Set the termination to a small number of generations, for example 10, until you are sure that the algorithm runs correctly.

Now you can run the algorithm with the `minimize` function in the same way as with the non-spatial problem. Add statements to print the design space (X) and the objective space (F) of all optimal individuals in your result object. In both of these variables, each row an individual while each column represents the control variable / objective value. Run the script to check whether the optimization runs completely without error for 50 generations. The result will look like in Figure 10.

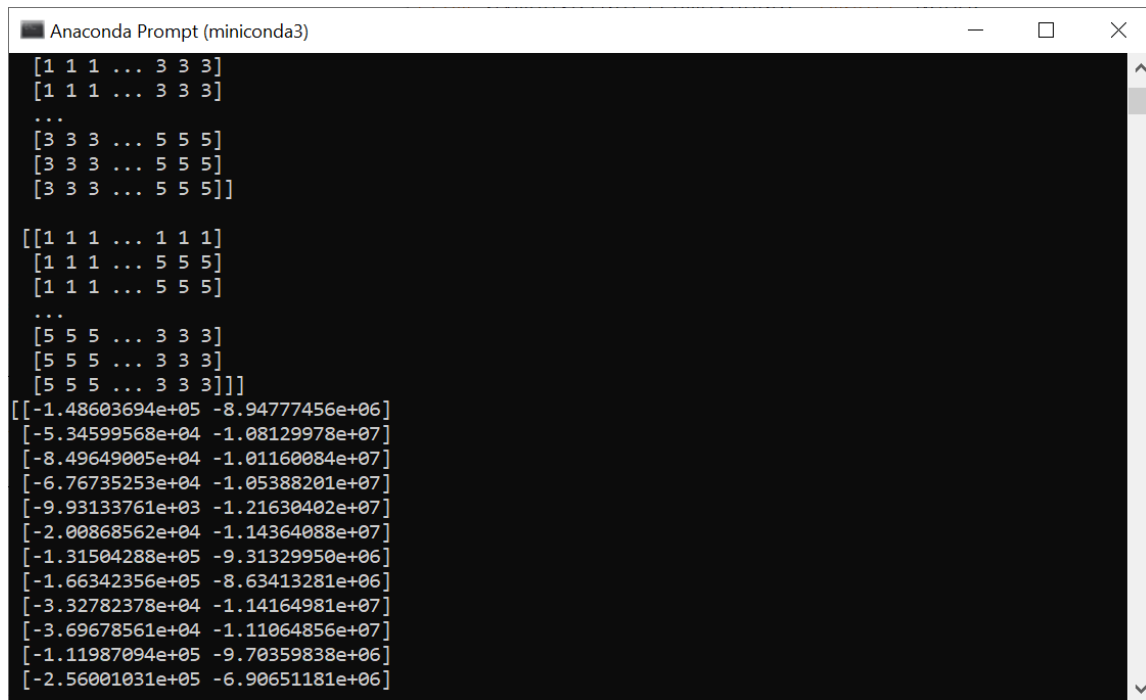


Figure 10: The X and F of the result object printed to the screen.

Note that NSGAI is a stochastic algorithm, which means that each time you run the algorithm you may get a different result. If you want, you can set the `seed` variable in the `minimize` method to a number, for example 1, to make sure you can reproduce your results. However, if the problem is implemented correctly, suitable parameters are chosen, and the number of generations is sufficient, the algorithm should converge to the same optimum despite this stochasticity.

10. Spatial optimization: visualize results

The next step is to visualize the results, as described below. In case expressions are not clear, look back at chapter 3. The example figures in this chapter are created with a population of 70, a mutation probability of 0.1 a point mutation probability of 0.1, and a termination criterion of 500 generations, but we encourage you to first test your script with fewer generations.

10.1 Objective space

Copy the code below to the `run_nsga2_spatial.py` file and run these lines to plot the Pareto front (Figure 11). If you want, you can save the plots to the disk with matplotlib `savefig`. Note the minus before the `res.F`. Why do we need the minus?

```

import matplotlib.pyplot as plt
from matplotlib.colors import ListedColormap
import matplotlib.patches as mpatches

f1, ax1 = plt.subplots(1)
im1 = plt.scatter(-res.F[:,0], -res.F[:,1])
ax1.set_title("Objective Space")
ax1.set_xlabel('Total yield [tonnes]')
ax1.set_ylabel('Above ground biomass [tonnes]')
f1.savefig(default_directory+"/outputs/objective_space.png", dpi=150)
plt.show()

```

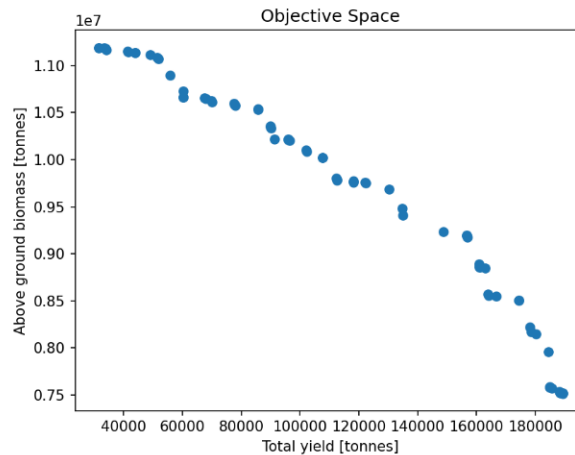


Figure 11. Pareto front (optimal individuals plotted in the objective space)

10.2 Optimized land use maps

Now plot the optimized land use maps using `matplotlib imshow`, as you did before. From `res.X`, get the land use map with the largest total yield and the one with the largest above ground biomass (i.e., the two extremes of the Pareto front) and plot these land use maps. Copy the code to your script and run it.

```
import matplotlib.patches as mpatches

# define the colors of the land use classes
cmap = ListedColormap(["#10773e", "#b3cc33", "#0cf8c1", "#a4507d", "#877712",
                       "#be94e8", "#eeefce", "#1b5ee4", "#614040", "#00000000"])

# build a legend with these colors and their land use label
legend_landuse = [mpatches.Patch(color="#10773e", label = 'Forest'),
                  mpatches.Patch(color="#b3cc33", label = 'Cerrado'),
                  mpatches.Patch(color="#0cf8c1", label = 'Secondary veg.'),
                  mpatches.Patch(color="#a4507d", label = 'Soy'),
                  mpatches.Patch(color="#877712", label = 'Sugarcane'),
                  mpatches.Patch(color="#be94e8", label = 'Fallow/cotton'),
                  mpatches.Patch(color="#eeefce", label = 'Pasture'),
                  mpatches.Patch(color="#1b5ee4", label = 'Water'),
                  mpatches.Patch(color="#614040", label = 'Urban'),
                  mpatches.Patch(color="#00000000", label = 'No data')]

# fetch the two extremes of the Pareto front from res.X
landuse_max_yield = res.X[np.argmax(-res.F[:,0], axis=0)]
landuse_max_biomass = res.X[np.argmax(-res.F[:,1], axis=0)]

# Plot them next to each other
f2, (ax2a, ax2b) = plt.subplots(1,2, figsize=(9,5))
im2a = ax2a.imshow(landuse_max_yield, interpolation='None',
                  cmap=cmap, vmin=0.5, vmax=10.5)
ax2a.set_title('Landuse map \nmaximized total yield', fontsize=10)
ax2a.set_xlabel('Column #')
ax2a.set_ylabel('Row #')
im2b = ax2b.imshow(landuse_max_biomass, interpolation='None',
                  cmap=cmap, vmin=0.5, vmax=10.5)
ax2b.set_title('Landuse map \nminimized CO2 emissions', fontsize=10)
ax2b.set_xlabel('Column #')
plt.legend(handles=legend_landuse, bbox_to_anchor=(1.05, 1), loc=2,
          prop={'size': 9})
```

```
# Adjust location of the plots to make space for legend and save
plt.subplots_adjust(right = 0.6, hspace=0.2)
plt.savefig(default_directory+"/outputs/landuse_max.png", dpi=150)
```

Compare the two land use maps (see our output in Figure 12) and consider whether they show what you expected. Based on these land use maps, do you think the algorithm found the real optimum?

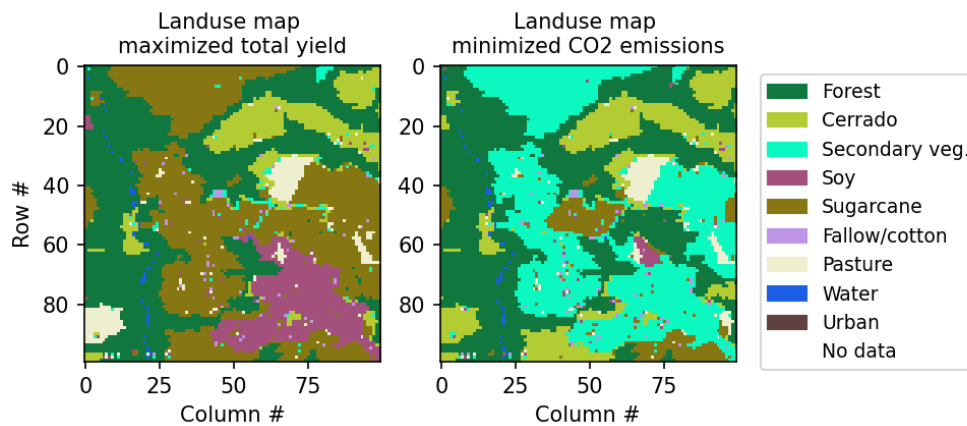


Figure 12. Land use maps of maximized total yield (left) and minimized CO_2 emissions (right).

10.3 Convergence

In optimization, we should not only look at the final result, but also analyse the algorithm's behaviour to make sure the algorithm converges and the number of generations is sufficient. When the algorithm converges, it has found a solution that stays stable over the generations.

Pymoo saves the properties of each calculated generation in the `history` variable of the result object. Obtain the objective values from each generation from the `res.history` using the code below. You can print `F` to see its structure.

```
# create an empty list to save objective values per generation
F = []
# iterate over the generations
for generation in res.history:
    # retrieve the optima for all objectives from the generation
    opt = generation.opt
    this_f = opt.get("F")
    F.append(this_f)

n_gen = np.array(range(1, len(F)+1))
print(n_gen)
```

There are several methods to check whether the algorithm converges. The first way is to plot the maximum value of both objective functions for each generation against the generation number. This way, you check whether the extremes improve and then become stable. Copy the code below to the 'run_nsga2_spatial.py' file and run it.

```
# get maximum (extremes) of each generation for both objectives
obj_1 = []
obj_2 = []
```

```

for i in F:
    max_obj_1 = min(i[:,0])
    max_obj_2 = min(i[:,1])

    obj_1.append(max_obj_1)
    obj_2.append(max_obj_2)

# visualize the maxima against the generation number
f3, (ax3a, ax3b) = plt.subplots(1,2, figsize=(9,5))
ax3a.plot(n_gen, -np.array(obj_1))
ax3a.set_xlabel("Generation")
ax3a.set_ylabel("Maximum total yield [tonnes]")
ax3b.plot(n_gen, -np.array(obj_2))
ax3b.set_xlabel("Generation")
ax3b.set_ylabel("Above ground biomass [tonnes]")
plt.subplots_adjust(wspace=0.25)
plt.savefig(default_directory+"/outputs/objectives_over_generations")

```

With the code above, the maximum above ground biomass and maximum total yield are plotted over the generations (Figure 13). Do you think the maximum values are stable after 500 generations?

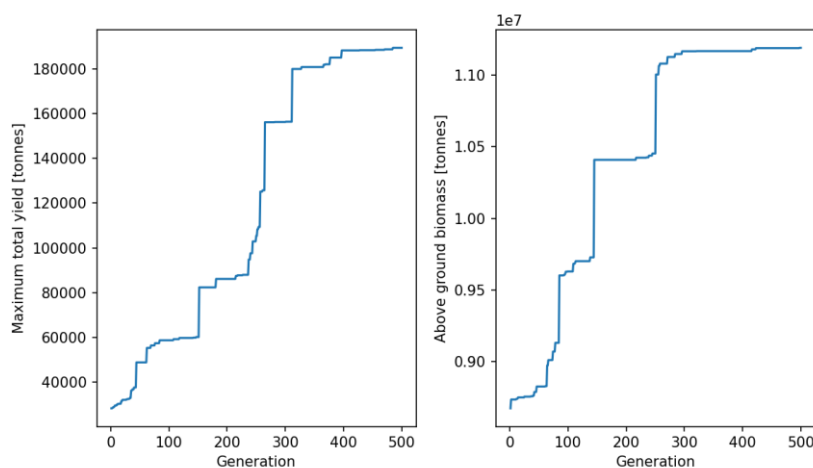


Figure 13. Maximum total yield (left) and above ground biomass (right) over the generations.

Another way to check for convergence is to plot the pareto front for various generations in a single figure. You can then see how much the pareto front changes between the generations and, as such, whether it becomes stable. Copy the code below and run it to plot the Pareto front for a chosen subset of generations. Study the plot (see our plot in Figure 14) to check for convergence.

```

# add here the generations you want to see in the plot
generations2plot = [..., ..., ...]

# make the plot
fig4, ax4 = plt.subplots(1)
# i - 1, because generation 1 has index 0
for i in generations2plot:
    plt.scatter(-F[i-1][:,0], -F[i-1][:,1])
ax4.set_xlabel('Total yield [tonnes]')
ax4.set_ylabel('Above ground biomass [tonnes]')
plt.legend(list(map(str, generations2plot)))
plt.savefig(default_directory+"/outputs/pareto_front_over_generations.png")

```

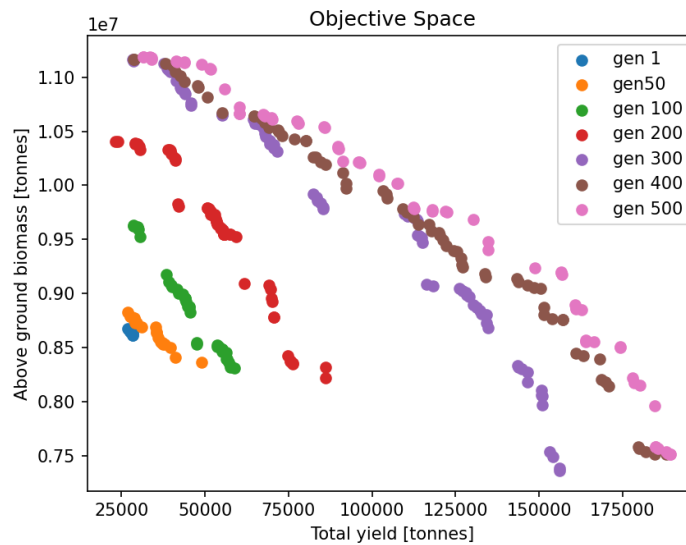


Figure 14. Pareto front over the generations.

The last way to check the convergence is by calculating the hypervolume (Figure 15), which is, for a two-objective case, the area between a certain reference point and the pareto front. In our case, the reference point is 0 yield and 0 carbon stock). The hypervolume grows as the population is optimized (because the objectives are maximized) and it remains stable when the pareto front is stable.

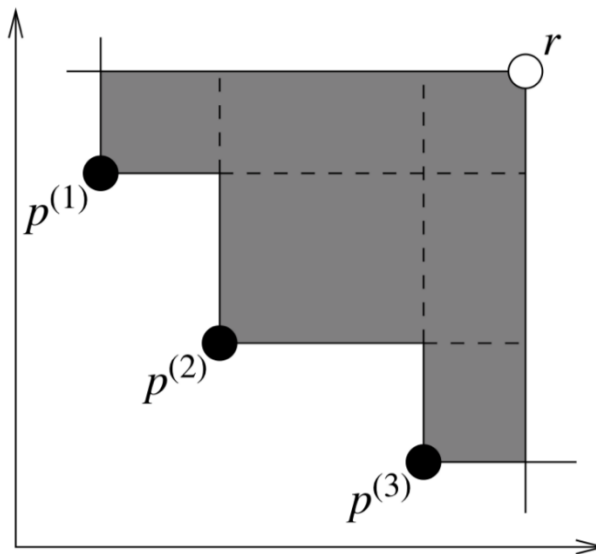


Figure 15. Illustration of the hypervolume. Source: Fonseca et al. (2006).

Run the code below to plot the hypervolume against the generation number and determine whether the algorithm converges (Figure 16).

```
from pymoo.indicators.hv import HV

# set reference point
ref_point = np.array([0.0, 0.0])
# create the performance indicator object with reference point
metric = HV(ref_point=ref_point)
# calculate for each generation the HV metric
```

```

hv = [metric.calc(f) for f in F]

# visualize the convergence curve
fig5, ax5 = plt.subplots(1)
ax5.plot(n_gen, hv, '-o', markersize=4, linewidth=2)
ax5.set_xlabel("Generation")
ax5.set_ylabel("Hypervolume")
plt.savefig(default_directory+"/outputs/hypervolume.png")

```

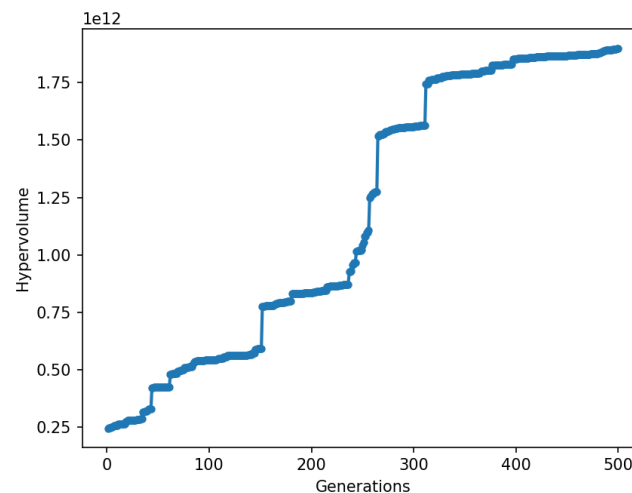


Figure 16. Hypervolume over the generations.

Now that you have looked at the optimization algorithm results from different perspectives, do you think the algorithm converges? If not, play around with the number of generations, population size, and mutation probabilities to see how it affects the results. Note that due to the large size of the study area, it might take a very large number of generations before the maxima are found. For this tutorial it is not necessary to find the exact optima, but make sure that you get a feeling of how the algorithm behaves under different parameter settings.

11. Things to think about for your own case study

This is the end of the tutorial and you have successfully optimized a land use map in Brazil with the NSGA-II algorithm. Of course, this is only one example of an optimization algorithm and there are many other, and probably more sophisticated, ways to initialize the land use raster and to perform crossover and mutation. As you can see from the results, only single cells are changed during the initialization. Think of ways you can change blocks of cells at once. In addition, entire patches are changed during the mutation, but maybe you only want to change part of the patches. You can also think about adding other objective functions to take, for example, biodiversity or landscape fragmentation (e.g. number of patches) into account. Good luck!

References

- Blank, J., & Deb, K. (2020). pymoo: Multi-objective Optimization in Python. *IEEE Access*, 8.
- Câmara, Gilberto; Picoli, Michelle; Maciel, Adeline; Simoes, Rolf; Santos, Lorena; Andrade, Pedro R; Ferreira, Karine; Begotti, Rodrigo; Sanches, Ieda; Carvalho, Alexandre X Y; Coutinho, Alexandre; Esquerdo, Julio; Antunes, Joao; Arvor, Damien (2019): Land cover change maps for Mato Grosso State in Brazil: 2001-2017 (version 3). *PANGAEA*.
- Deb, K., Pratap, A., Agarwal, S., & Meyarivan, T. (2002). NSGA-II. *IEEE transactions on evolutionary computation*, 6(2).
- Eggleston, S., Buendia, L., Miwa, K., Ngara, T., & Tanabe, K. (Eds.). (2006). 2006 IPCC guidelines for national greenhouse gas inventories. *Hayama, Japan: Institute for Global Environmental Strategies*. Volume 4: Agriculture, Forestry and Other Land Use.
- Fischer, G., Nachtergaele, F. O., Prieler, S., Teixeira, E., Tóth, G., Van Velthuisen, H., Verelst, L. & Wiberg, D. (2012). Global agro-ecological zones (GAEZ v3.0)-model documentation.
- Fonseca, C. M., Paquete, L., & López-Ibáñez, M. (2006). An improved dimension-sweep algorithm for the hypervolume indicator. *IEEE international conference on evolutionary computation*, 1157-1163.
- Kerdan, I. G., Giarola, S., Jalil-Vega, F., & Hawkes, A. (2019). Carbon sequestration potential from large-scale reforestation and sugarcane expansion on abandoned agricultural lands in Brazil. *Polytechnica*, 2(1), 9-25.
- Ribeiro, S. C., Fehrmann, L., Soares, C. P. B., Jacovine, L. A. G., Kleinn, C., & de Oliveira Gaspar, R. (2011). Above-and belowground biomass in a Brazilian Cerrado. *Forest Ecology and Management*, 262(3), 491-499.
- Strauch, M., Cord, A. F., Pätzold, C., Lautenbach, S., Kaim, A., Schweitzer, C., ... & Volk, M. (2019). Constraints in multi-objective optimization of land use allocation—Repair or penalize? *Environmental Modelling & Software*, 118, 241-251.